

luaPilot

luaPilot est un binaire qui fournit des fonctionnalités avancées : découpage de chaînes, fusion de tables, listing de fichiers avec ou sans itérateur, calculs de hash, et bien plus. Écrit en C++23 pour des performances optimales, il permet d'embarquer un fichier ZIP dans l'exécutable si on le souhaite. Lorsque le binaire est lancé, les fichiers embarqués (`main.lua` et tous les modules chargés par `require`) sont lus à la demande directement depuis le ZIP appendu.

Il est actuellement compilé pour Lua 5.5.0, mais on peut changer la version en modifiant `build_local.sh` puis en recompilant.

Voir les **exemples** pour aller plus loin...

Télécharger la dernière version

Pour télécharger la dernière version de luaPilot, rendez-vous sur la page des releases sur GitHub et téléchargez la version la plus récente.

Compilation

Pour compiler le projet :

1. **Cloner le dépôt :**

```
git clone https://github.com/Chipsterjulien/luapilot_standalone.git
cd luapilot_standalone
```

2. **Compiler localement :**

```
./build_local.sh
```

Le script de build télécharge et compile ses propres dépendances (Lua, OpenSSL, miniz). Les seuls prérequis sur votre système sont un compilateur C++23, CMake, wget et unzip.

Utilisation

Le point d'entrée est `main.lua`. Vous pouvez charger d'autres fichiers Lua depuis ce dernier via `require`.

```
./luapilot .
```

Si un ZIP est embarqué :

```
./luapilot_with_zip_embedded
```

Créer un exécutable depuis un script Lua via `--create-exe`

Si luaPilot est installé globalement :

```
# exemple :
echo 'luapilot.helloThere()' > main.lua
luapilot --create-exe . luapilot_with_lua_script_embedded
./luapilot_with_lua_script_embedded
```

Si luaPilot n'est pas installé globalement :

```
# exemple :
echo 'luapilot.helloThere()' > main.lua
./luapilot --create-exe . luapilot_with_lua_script_embedded
./luapilot_with_lua_script_embedded
```

Dans cet exemple, luaPilot construit un ZIP depuis le répertoire courant puisque `.` est le premier argument.

Pour utiliser votre propre ZIP :

```
cat luapilot my_file.zip > my_new_exec_program
chmod +x my_new_exec_program
./my_new_exec_program
```

Modèle de sécurité

LuaPilot exécute des scripts Lua de confiance. Ce n'est pas une sandbox.

Les scripts s'exécutent avec la bibliothèque standard Lua complète (`luaL_openlibs`), et LuaPilot expose en plus des primitives système comme `exec`, `remove`, `rmdirAll` et `chdir`. Un script a donc les mêmes privilèges que le processus qui le lance : il peut lire, modifier ou supprimer des fichiers, et lancer des commandes arbitraires. C'est volontaire — comme `make`, un script shell, ou n'importe quel outil de build, LuaPilot est conçu pour exécuter du code que vous contrôlez.

N'exécutez que des `main.lua` et des modules `require` auxquels vous faites confiance. **N'utilisez pas** LuaPilot pour exécuter du Lua provenant de sources non fiables ; il n'offre aucune isolation contre du code hostile, et n'est pas pensé pour. Si vous devez exécuter des scripts non fiables, utilisez plutôt une sandbox Lua dédiée à cet usage.

Le travail de durcissement effectué dans LuaPilot (gestion des chemins et symlinks, cleanup des groupes de processus, limites de sortie, checksums des dépendances) protège les usages *légitimes* contre les accidents et les attaques de supply chain. Il ne transforme pas LuaPilot en barrière contre des scripts malveillants, et ne doit pas être considéré comme telle.

Quelques exemples

Hello there

```
luapilot.helloThere()
```

Manipulation de fichiers

```
-- Lister les fichiers d'un répertoire
local files, err = luapilot.listFiles(".")
if err then
    print(err)
else
    for _, file in ipairs(files) do
        print(file)
    end
end

-- Vérifier qu'un fichier existe
local exists, err = luapilot.fileExists("my/folder/file.txt")
if err then
    print(err)
else
    print("File exists: " .. tostring(exists))
end
```

Manipulation de tables

```
local inspect = require('inspect')
-- Fusionner deux tables
local table1 = {a = 1, b = 2}
local table2 = {b = 3, c = 4}
local mergedTable = luapilot.mergeTables(table1, table2)
print(inspect(mergedTable))

-- Fusionner plusieurs tables
local mergedMultipleTables = luapilot.mergeTables(table1, table2, {d = 5}, {e = 6})
print(inspect(mergedMultipleTables))

-- Copie profonde d'une table
local t4 = { i = 8, j = { k = 9, l = 10, m = { n = 11, o = 12 } } }
local deepCopy = luapilot.deepCopyTable(t4)
print(inspect(deepCopy))
```

Manipulation de chaînes

```
-- Découper une chaîne
local parts = luapilot.split("hello,world", ",")
for _, part in ipairs(parts) do
    print(part)
end
```

JSON

```
-- Encoder (compact, ou indenté avec opts.indent)
local s, err = luapilot.json.encode({ name = "ada", tags = { "x", "y" } })
local pretty = luapilot.json.encode({ a = 1 }, { indent = 2 })

-- Décoder
local value, derr = luapilot.json.decode('{ "n": 42, "ok": true }')

-- Sentinelles :
--   luapilot.json.null      <-> JSON null (distinct de nil :
--                               aller-retour sans perte de clé)
--   luapilot.json.empty_array -> force [] (une table vide {} donne {})
local doc = { items = luapilot.json.empty_array, missing = luapilot.json.null }

-- as_array : force une table à être sérialisée comme array, même vide.
-- Utile pour un tableau construit dynamiquement qui peut finir vide.
local tags = {}
-- ... remplissage conditionnel de tags ...
local s2 = luapilot.json.encode({ tags = luapilot.json.as_array(tags) })
-- tags resté vide -> {"tags":[]} (et non {"tags":{}})
```

`as_array(t)` renvoie `t` (chaînable) et le marque via sa métatable — elle écrase donc une métatable préexistante sur `t` (sans conséquence pour des tables de données, le seul cas où l'on sérialise du JSON).

Une table aux clés `1..n` devient un tableau JSON ; une table à clés string devient un objet. Les clés mixtes, les tableaux à trous, NaN / Infinity et les valeurs non encodables renvoient `(nil, err)` — jamais d'exception Lua.

Voir les exemples pour aller plus loin...

Client HTTP

`luapilot.http` est un client HTTP/HTTPS. HTTPS réutilise l'OpenSSL déjà lié à LuaPilot — pas de dépendance TLS supplémentaire.

Son modèle d'erreur suit celui de `luapilot.exec` :

- une **erreur de transport** (DNS, connexion refusée, TLS, timeout, ...) renvoie `(nil, "http: <raison>")`. La chaîne de raison est le label d'erreur brut de `cpp-httpplib` — par exemple `"http: Timeout"` sur un timeout global, `"http: Connection"` sur une connexion refusée. Aucune heuristique temporelle ne la renomme ;
- un **échange terminé renvoie (result, nil) quel que soit le statut** — un 404 ou 500 n'est *pas* une erreur, exactement comme un code de retour non nul n'est pas une erreur pour `exec`. Le statut est dans `result.status` ;
- un **mauvais usage** (url manquante, mauvais *type* d'option) lève une erreur Lua, comme les autres bindings LuaPilot. Une *valeur* d'option mauvaise (méthode inconnue, URL malformée, `timeout <= 0`, body sur GET/HEAD/OPTIONS) renvoie `(nil, err)` — pas de comportement silencieux.

```
-- GET simple
local res, err = luapilot.http.get("https://example.com")
if not res then
    print("request failed: " .. err)      -- erreur de transport
else
    print(res.status)                    -- ex. 200, 404, 500
    print(res.body)                      -- binary-safe (NUL ok)
    print(res.headers["content-type"])   -- clés des headers en minuscules
end
```

```
-- POST avec un body
local r = luapilot.http.post("https://api.example.com/items",
  '{"name":"ada"}',
  { headers = { ["Content-Type"] = "application/json" } })

-- Forme générique
local r2, e2 = luapilot.http.request({
  url      = "https://example.com/search",
  method   = "GET",
  query    = { q = "lua http", page = 2 }, -- percent-encodés
  headers  = { Accept = "text/html" },
  timeout  = 5,                          -- secondes (de bout en bout)
})
```

Options de request(opts)

Champ	Type	Défaut	Signification
url	string	— (requis)	http:// ou https://
method	string	"GET"	GET/HEAD/OPTIONS/POST/PUT/PATCH/DELETE
headers	table	{}	{ ["Name"] = "value" }
body	string	—	corps de la requête (méthodes à body uniquement ; le passer sur GET/HEAD/OPTIONS renvoie (nil, err))
query	table	—	{ k = v } → ?k=v&..., percent-encodés
timeout	number	aucun	secondes (> 0) ; temps max de bout en bout, équivalent à --max-time de curl (appliqué via set_max_timeout cpp-httpplib)
verify	bool	true	vérification du certificat serveur TLS
ca_cert	string	—	chemin vers un bundle de CA
follow_redirects	bool	false	non suivis sauf demande explicite

get(url [, opts]) et post(url [, body] [, opts]) sont de simples wrappers autour de request (un seul chemin de code).

Table result

Champ	Type	Notes
status	integer	statut HTTP (toute valeur, y compris 4xx/5xx)
body	string	binary-safe (les octets NUL embarqués sont préservés)
headers	table	clés en minuscules (HTTP est case-insensitive, Lua non) ; en cas de doublons, la dernière valeur écrase

Hors v1 (additif possible plus tard sans casser le SemVer) : streaming, multipart, cookies, sessions, keep-alive exposé. Il n'y a **pas de serveur HTTP** dans l'API LuaPilot.

Voir les exemples pour aller plus loin...

argparse — parseur d'arguments en ligne de commande

argparse est un module Lua pur bundlé dans le binaire LuaPilot, chargé via require("argparse"). Il construit un parseur, parse arg (ou une table explicite), et renvoie un seul contrat (res, err) — pas de print implicite, pas de os.exit. Le script garde toujours le contrôle.

```
local argparse = require("argparse")

local p = argparse("myprog", "do something useful")
:flag("-v --verbose", { help = "verbose output" })
:option("-o --output", { default = "a.out", help = "output file" })
:option("-m --mode", { choices = { "fast", "safe" } })
:argument("input", { help = "input file" })
```

```

local res, err = p:parse()      -- lit `arg[1..n]` par défaut
if not res then
    io.stderr:write(err, "\n")  -- saisie utilisateur invalide
    os.exit(2)
elseif res.help then
    io.write(res.usage)         -- -h / --help est un succès
    os.exit(0)
end

-- cas normal : res.input, res.output, res.verbose, res.mode

```

Trois formes de retour pour `parse()`, toutes sous le même contrat (`res`, `err`) :

Cas	res	err
Succès	{ dest = value, ... }	nil
-h / --help	{ help = true, usage = "<text>" }	nil (l'aide est un succès, jamais une erreur)
Mauvaise saisie	nil	"unknown option '--nope'", "missing required argument 'input'..."

`parse()` ne lève jamais d'erreur Lua sur la saisie utilisateur — chaque échec runtime est remonté via (`nil`, `err`). Les seules erreurs levées sont des fautes de *programmeur* dans le builder (spec vide, nom d'option sans -, doublon de nom), équivalents à un `luaL_error` côté binding C.

Méthodes du builder (chacune renvoie `self`, elles sont donc chaînables) :

Méthode	Rôle
<code>p:flag("-v --verbose", opts)</code>	flag booléen, vaut false par défaut
<code>p:option("-o --output", opts)</code>	option qui prend une valeur
<code>p:argument("name", opts)</code>	argument positionnel
<code>p:parse([t])</code>	parse <code>arg[1..n]</code> par défaut, ou la table <code>t</code>
<code>p:get_usage()</code>	texte d'aide (également exposé via <code>res.usage</code> sur <code>-h</code>)

Spec d'option : noms court et long séparés par des espaces ("`-o --output`"), au moins un doit commencer par -. Le nom de destination vient de la première forme longue (`--output` → `res.output`), avec fallback sur la première forme courte ; `opts.dest` peut écraser explicitement.

Table opts (tous les champs optionnels) :

Champ	S'applique à	Signification
<code>default</code>	option, argument	valeur si absent ; un positionnel avec default devient optionnel
<code>required</code>	option, argument	force la présence (un positionnel est requis par défaut sauf si default est posé)
<code>choices</code>	option, argument	liste de valeurs autorisées, testée sur la chaîne brute
<code>convert</code>	option, argument	fonction (<code>raw</code>) → <code>value</code> appliquée après <code>choices</code> ; renvoyer <code>nil</code> ou lever est remonté via (<code>nil</code> , <code>err</code>)
<code>help</code>	tous	texte utilisé dans <code>get_usage()</code> / <code>res.usage</code>
<code>dest</code>	option	force la clé de résultat

Note sur choices et convert : `choices` est vérifié sur la chaîne brute avant que `convert` ne tourne. Si on combine les deux, exprimer `choices` en chaînes — par exemple `choices = {"1", "2", "3"}` avec `convert = tonumber` — pas avec les valeurs converties.

Source des arguments. `parse()` sans argument lit le `arg` global aux indices `1..n` (les vrais arguments utilisateur, identiques sur les trois modes de lancement LuaPilot — package, dossier, embarqué via PATH). Les

indices ≤ 0 (chemin binaire, nom de dossier) sont intentionnellement ignorés. On peut aussi passer une table explicite : `p:parse({"-v", "input.txt"})`.

Formes d'arguments supportées : `--long value`, `--long=value`, `-s value`, `-s=value`. Un token `--` termine le parsing d'options — tout token après est traité comme positionnel (donc `-h` après `--` est la chaîne littérale `"-h"`, pas le flag d'aide).

Hors v1 (additif possible plus tard, Lua pur — pas de cassure SemVer) : sous-commands, **nargs** variadique, formes courtes collées (`-abc`, `-fvalue`).

Voir les exemples pour aller plus loin...

logging — logger à niveaux

`logging` est un module Lua pur bundlé dans le binaire `LuaPilot`, chargé via `require("logging")`. Il fournit du logging à niveaux avec horodatage, un sink de sortie configurable, et des couleurs ANSI optionnelles — tout en restant petit, prévisible, et sans effet de bord sauf si vous demandez explicitement de la sortie.

```
local log = require("logging")

log.info("user=", uid, " connected")      -- variadique comme print
log.warn("slow query (", elapsed_ms, "ms)")
log.error("connection refused:", host)

log.set_level("debug")                    -- "trace"/"debug"/... ou log.DEBUG
log.set_output(io.open("app.log", "a"))   -- tout objet ayant :write
log.set_color(true)                       -- ANSI OFF par défaut, opt-in
```

Format de sortie (figé en v1, pas encore de formater personnalisable) :

```
2026-05-22 14:32:01 [INFO ] user= 42  connected
```

L'étiquette de niveau est paddée à 5 caractères pour que les colonnes restent alignées (`[TRACE]`, `[DEBUG]`, `[INFO]`, `[WARN]`, `[ERROR]`).

Niveaux ($\geq$ seuil passe, les autres sont silencieusement abandonnés — c'est le seul silence légitime, puisqu'il a été demandé) :

Nom	Constante	Numérique
trace	<code>log.TRACE</code>	10
debug	<code>log.DEBUG</code>	20
info (défaut)	<code>log.INFO</code>	30
warn	<code>log.WARN</code>	40
error	<code>log.ERROR</code>	50

API

Fonction	Rôle
<code>log.trace(...)</code> / <code>log.debug(...)</code> / <code>log.info(...)</code> / <code>log.warn(...)</code> / <code>log.error(...)</code>	émettre un message ; les args variadiques sont convertis avec <code>tostring</code> et joints par des espaces (comme <code>print</code>)
<code>log.set_level(level)</code>	accepte une string (<code>"info"</code> , case-insensitive) ou un nombre (<code>log.INFO</code>)
<code>log.set_output(sink)</code>	tout objet répondant à <code>:write</code> (défaut : <code>io.stderr</code>)
<code>log.set_color(on)</code>	booléen strict ; défaut <code>false</code>
<code>log.get_level()</code> / <code>log.get_output()</code> / <code>log.get_color()</code>	introspection

Défauts

- Niveau : **info** (donc `trace` et `debug` sont droppés tant qu'on ne les demande pas)
- Sortie : **io.stderr** (convention Unix — garde `print()` et autres données `stdout` non contaminés par les diagnostics)

- Couleur : **false** (opt-in explicite ; pas d'auto-détection en v1)

Couleurs (quand `set_color(true)`) :

Niveau	Couleur
trace	dim (gris)
debug	cyan
info	aucune (neutre — info est le flux par défaut, sans urgence)
warn	jaune
error	rouge

Contrat d'erreur

- Les cinq fonctions d'émission (`log.trace ... log.error`) **ne lèvent jamais et ne renvoient rien**. Si `:write` échoue (disque plein, handle fermé, sink qui lève), l'échec est silencieusement avalé. C'est la règle standard du logger : une ligne de log ratée ne doit pas se transformer en crash de logique métier.
- Les setters (`set_level`, `set_output`, `set_color`) **lèvent via `error()` en cas de mauvais usage** (nom de niveau inconnu, sortie non writable, couleur non booléenne) — ce sont des fautes de programmeur, équivalents à un `luaL_error` côté binding C.

Hors v1 (tous additifs plus tard, Lua pur — pas de cassure SemVer) : rotation des fichiers, loggers nommés, hiérarchie et propagation, filtres personnalisés, formats personnalisables, millisecondes ou UTC dans les timestamps, détection auto du TTY (`isatty` opt-in).

Voir les exemples pour aller plus loin...

sys — utilitaires système

`luapilot.sys` expose un petit ensemble d'utilitaires système, mais attaché **directement sous `luapilot.*`** (pas de sous-table `sys` imbriquée) — ils côtoient `exec`, `sleep`, `currentDir`, etc. La liste est courte et ce sont des bases de scripting, le plat layout reste donc simple.

```
print(luapilot.hostname())           -- "my-laptop"
print(luapilot.pid())                -- 12345

local sh = luapilot.which("sh")      -- "/usr/bin/sh"
local missing, err = luapilot.which("nope_42") -- nil, "which: 'nope_42' not found in PATH"

local port = luapilot.env("PORT") or "8080" -- idiome env-or-default

local ok_set, err = luapilot.setenv("LANG", "C")
if not ok_set then io.stderr:write(err, "\n") end

local u = luapilot.uname()           -- { sysname, nodename, release, version, machine }
print(u.sysname, u.machine)          -- "Linux", "x86_64"
```

Fonctions

Fonction	Renvoie	Notes
<code>which(name)</code>	"path" (nil, err)	Cherche un exécutable dans \$PATH. Si <code>name</code> contient /, le chemin est testé directement (mimétisme <code>/usr/bin/which</code>). Renvoie un chemin absolu.
<code>env(name)</code>	"value" nil	Lit une variable d'environnement. L'absence est nil seul (pas de message d'erreur), pour permettre l'idiome <code>or default</code> . Un mauvais type lève.
<code>setenv(name, value)</code>	(true, nil) (nil, err)	Écrit une variable d'environnement (overwrite). Effet de bord process-global. Un <code>name</code> invalide (ex. contenant =) renvoie (nil, err).
<code>hostname()</code>	"host" (nil, err)	Wrap de <code>gethostname(2)</code> .
<code>uname()</code>	table (nil, err)	Wrap de <code>uname(2)</code> . La table contient <code>sysname</code> , <code>nodename</code> , <code>release</code> , <code>version</code> , <code>machine</code> — toutes des strings non vides (équivalents des champs <code>uname -a</code>).
<code>pid()</code>	integer	Wrap de <code>getpid(2)</code> , ne peut pas échouer (POSIX).

Contrat d'erreur (cohérent avec le reste de `LuaPilot`) :

- **Succès** — la valeur est renvoyée directement (pas de wrapping).
- **Échec runtime attendu** (`which("nope")`, `setenv("bad=name", ...)`) — (`nil`, `"msg"`).
- **Mauvais usage** (argument manquant, mauvais type) — lève via `luaL_error`.
- **env** est l'exception délibérée : une variable absente n'est **pas** une erreur, elle renvoie `nil` seul. Pour l'idiotisme `local p = luapilot.env("PORT") or "8080"`, qui est l'usage dominant des lectures d'`env`.

Note sur l'argument de `env(name)` : comme tout autre binding LuaPilot, `env` utilise `luaL_checkstring`, qui coerce silencieusement les *nombres* en leur forme string (convention Lua). Donc `env(42)` est équivalent à `env("42")` et renvoie la variable que "42" résout (probablement `nil`). Passez une table ou un booléen pour forcer une erreur Lua.

Hors v1 (tous additifs plus tard, additif — pas de cassure SemVer) : `unsetenv`, `getuid/getgid`, `getppid`, `getlogin`, dump complet de l'environnement.

Voir les exemples pour aller plus loin...

time — horloges monotone et temps réel

Deux fonctions pour mesurer des durées et timestamper des événements avec une précision sub-seconde — ce que la stdlib Lua ne donne pas.

```
-- Mesurer une durée (utilisez ÇA pour TOUTES les mesures de durée) :
local t0 = luapilot.monotonic()
faire_un_truc()
local elapsed = luapilot.monotonic() - t0    -- secondes avec partie fractionnaire
print(string.format("durée : %.3f s", elapsed))

-- Récupérer un timestamp wall-clock (logs, métadonnées fichier, comparaisons) :
local ts = luapilot.now()                    -- secondes depuis Unix epoch
print(os.date("!!Y-%m-%dT%H:%M:%SZ", ts))    -- "2026-05-26T14:32:11Z"
```

Fonction	Backend	Renvoie
<code>luapilot.monotonic()</code>	<code>clock_gettime(CLOCK_MONOTONIC)</code>	Temps depuis un point arbitraire (typiquement le boot). Ne saute jamais en arrière — fiable pour mesurer des durées.
<code>luapilot.now()</code>	<code>clock_gettime(CLOCK_REALTIME)</code>	Temps depuis l'Unix epoch (1970-01-01 UTC). Peut sauter (NTP, ajustement manuel) — uniquement pour timestamps wall-clock.

Pourquoi pas juste `os.time()` ou `os.clock()` ?

- `os.time()` renvoie un **entier** en secondes entières — pas de précision sub-seconde. Inutilisable pour des latences IRC ou des timestamps de logs précis.
- `os.clock()` mesure le **temps CPU** du processus, pas le temps mur — renvoie 0 quand le processus attend dans `sleep`, `recv`, etc.
- Aucun des deux ne distingue horloge monotone et temps réel. `os.time()` peut sauter en arrière sur correction NTP, cassant silencieusement les mesures de durée.

Le formatage reste dans la stdlib Lua. Utilisez `os.date("!!Y-%m-%dT%H:%M:%SZ", ts)` pour l'ISO 8601 UTC, `os.date("%c", ts)` pour le format locale, ou écrivez votre `format_duration(seconds)` en quelques lignes — pas besoin de les ajouter à LuaPilot.

signal — signaux POSIX

Trois fonctions pour réagir à des signaux comme `SIGTERM` (envoyé par `systemctl stop`), `SIGINT` (Ctrl-C) ou des signaux applicatifs comme `SIGUSR1`. Le cas d'usage typique est l'arrêt propre d'un script de longue durée — fermer les sockets, sauvegarder l'état sur disque, dire au revoir à un peer — avant que le processus ne se termine.

```
-- Arrêt propre sur systemctl stop / Ctrl-C
luapilot.signal.handle("TERM", function()
    log.info("SIGTERM reçu, arrêt propre")
    bot:disconnect()    -- p.ex. envoyer QUIT sur IRC
    -- db:close() si tu as une base
    os.exit(0)
end)
```



```

luapilot.signal.handle("INT", function()
    log.info("Ctrl-C, sortie")
    os.exit(0)
end)

-- Ignorer SIGPIPE : un write sur socket fermée renvoie EPIPE au
-- lieu de tuer le processus. La couche socket de LuaPilot le gère
-- déjà, mais c'est un défaut utile pour tout code qui fait de l'I/O.
luapilot.signal.ignore("PIPE")

-- Signal applicatif pour "recharger la config sans redémarrer"
luapilot.signal.handle("HUP", function()
    log.info("SIGHUP, rechargement de la config")
    config = reload_config()
end)

```

Fonction	Effet
<code>luapilot.signal.handle(name, fn)</code>	Installe un callback Lua pour le signal. Le callback est invoqué entre les opérations LuaPilot (jamais dans le handler C — voir plus bas).
<code>luapilot.signal.handle(name, nil)</code>	Désinstalle le callback et restaure le comportement système par défaut.
<code>luapilot.signal.ignore(name)</code>	Marque le signal comme ignoré (SIG_IGN). Le kernel le jette sans prévenir le processus.
<code>luapilot.signal.default(name)</code>	Restitue l'action système par défaut (SIG_DFL). Pour TERM/INT/HUP, ça tue le processus.

Les trois renvoient `true` en succès, `(nil, err)` en échec. Passer un nom de signal non supporté lève une erreur Lua.

Signaux supportés (liste blanche) :

Nom	Numéro	Usage typique
TERM	SIGTERM (15)	<code>systemctl stop</code> , demande d'arrêt propre
INT	SIGINT (2)	Ctrl-C depuis le terminal
HUP	SIGHUP (1)	Convention “recharger la config” (non imposé — c’est votre handler qui décide)
USR1	SIGUSR1 (10)	Libre, applicatif
USR2	SIGUSR2 (12)	Libre, applicatif
PIPE	SIGPIPE (13)	Usuellement ignoré pour que <code>write</code> sur socket fermée renvoie EPIPE au lieu de tuer le processus

Non supportés et pourquoi :

- KILL, STOP — non interceptables par aucun processus (garantie POSIX).
- SEGV, BUS, FPE, ILL — ces signaux indiquent une faute matérielle ou mémoire. Exécuter du Lua arbitraire depuis un handler dans cet état n’est pas sûr ; le programme est déjà cassé.
- CHLD — réservé pour un éventuel `exec` async futur ; pas exposé aujourd’hui pour garder l’API minimale.
- ALRM — entrerait en conflit avec la mécanique interne de `sleep`.

Interruption des syscalls bloquants Quand un signal géré arrive pendant une opération LuaPilot bloquante (`socket:recv`, `socket:recv_line`, `socket:send`, `socket:accept`, `socket:connect`, `socket:connect_tls`, `luapilot.sleep`), l’opération renvoie immédiatement `(nil, "interrupted")` et le callback Lua s’exécute avant le retour. C’est ce qui permet d’interrompre un long `recv_line()` ou un `sleep(60)` sans attendre l’expiration du timeout.

```

luapilot.signal.handle("USR1", function()
    print(">>> signal reçu")
end)

```

```
end)
```

```
local ok, err = luapilot.sleep(30, "s")    -- renvoie normalement après 30s
-- Envoyez : kill -USR1 <pid> pendant qu'il dort
-- → callback exécuté, puis :
-- ok = nil
-- err = "interrupted"
```

Pour `socket:recv_line`, les octets déjà accumulés avant l'interruption sont **conservés** pour le prochain appel à `recv_line` — même logique que sur `timeout`. Aucune donnée perdue.

Les signaux **non** gérés par `luapilot.signal` (par exemple `SIGWINCH` lors d'un redimensionnement de terminal) conservent le comportement standard de `retry EINTR` en interne : un événement cosmétique n'interrompra pas un long `recv()`.

Workers Les workers (`luapilot.workers.spawn`) bloquent automatiquement tous les signaux supportés via `pthread_sigmask` au démarrage du thread. Cela garantit que les callbacks Lua ne s'exécutent que dans le thread principal, quel que soit le thread choisi par le kernel pour délivrer le signal. Vous installez vos handlers dans le script principal ; les workers ne voient pas les signaux.

Ce qui tourne dans le callback Les handlers de signal POSIX doivent être “async-signal-safe” : pas d'allocation, pas d'appel à la plupart des fonctions de la `libc`, pas d'interaction avec un runtime comme celui de Lua. LuaPilot impose cette règle en **n'exécutant pas votre callback Lua dans le handler C**. Le handler C fait le strict minimum (positionner un drapeau) et votre callback s'exécute plus tard dans un contexte normal, depuis un de ces endroits :

- immédiatement au retour d'un syscall bloquant interrompu (le cas “`interrupted`” ci-dessus) ;
- sinon, depuis un debug hook Lua qui se déclenche toutes les ~10 000 instructions Lua — soit quelques millisecondes de latence typique.

Conséquence pratique : les callbacks **peuvent** faire tout ce que du code Lua normal fait (allouer, appeler des fonctions Lua, logger, appeler `os.exit`, écrire des fichiers). Ils **devraient** rester courts et ne pas bloquer longtemps — sinon les signaux qui s'accumulent attendront.

Les erreurs levées par le callback sont avalées silencieusement par `pcall` : un handler bogué ne doit pas faire crasher le programme. Si vous voulez de la visibilité côté erreur, encapsulez le corps du callback dans votre propre `pcall` et loggez explicitement.

sqlite — base de données SQL embarquée

Un wrapper haut niveau autour d'une SQLite 3.53.1 embarquée (liée statiquement, aucune dépendance système). Deux méthodes couvrent l'essentiel : `db:exec` pour les DDL/DML et `db:query` pour itérer les `SELECT`. Le binding reste compact et idiomatique Lua ; si vous avez besoin d'un contrôle au niveau `prepare/bind/step/finalize`, un binding SQLite plus bas niveau sera sans doute plus approprié.

```
local db = luapilot.sqlite.open("bot.db", {
    wal = true,          -- un writer + plusieurs readers en parallèle
    busy_timeout = 5000, -- retry jusqu'à 5s sur conflit de verrou
})

db:exec([[
    CREATE TABLE IF NOT EXISTS users (
        id    INTEGER PRIMARY KEY,
        name  TEXT NOT NULL,
        age   INTEGER,
        bio   TEXT
    )
]])

-- INSERT avec bind positionnel
db:exec("INSERT INTO users (name, age) VALUES (?, ?)", { "alice", 30 })

-- INSERT avec bind nommé (:, @, $ tous acceptés)
db:exec("INSERT INTO users (name, age) VALUES (:n, :a)",
```

```
{ n = "bob", a = 25 })
```

```
-- SELECT : itérateur lazy, une ligne à la fois
for row in db:query("SELECT id, name, age FROM users WHERE age > ?", { 20 }) do
    print(row.id, row.name, row.age)
end

db:close()
```

Fonction / méthode	Retour
luapilot.sqlite.open(path, opts?)	db en succès, (nil, err) en échec
db:exec(sql, params?)	(true, nil) ou (nil, err)
db:query(sql, params?)	un itérateur callable, ou (nil, err)
db:close()	(true, nil) ; idempotent
iter:close()	(true, nil) ; idempotent ; libère le statement tôt

Le userdata `db` est finalisé automatiquement par `__gc` si vous oubliez `close()`, mais un `close` explicite est recommandé pour les programmes longue durée.

open : chemins et options `path` est soit `":memory:"` pour une base en RAM (perdue au `close`), soit un chemin de fichier (créé s'il n'existe pas).

`opts` est une table avec deux clés optionnelles :

Clé	Type	Défaut	Effet
<code>wal</code>	boolean	false	Émet <code>PRAGMA journal_mode=WAL</code> . WAL permet readers + un writer en parallèle, c'est le mode recommandé pour la plupart des apps ; fallback silencieux vers un autre mode pour les bases <code>:memory:</code> .
<code>busy_timeout</code>	integer ms	0	Configure <code>sqlite3_busy_timeout</code> . Quand une autre connexion détient un verrou, SQLite retry transparent jusqu'à N ms avant de retourner <code>SQLITE_BUSY</code> . Recommandé : 1000-5000 pour les apps multi-connexions.

exec : DDL et DML `db:exec(sql)` sans params passe par `sqlite3_exec` et accepte plusieurs statements séparés par `;` :

```
db:exec([[
    CREATE TABLE t (a, b);
    CREATE INDEX idx_t_a ON t(a);
    INSERT INTO t VALUES (1, 'one');
]])
```

`db:exec(sql, params)` compile un seul statement, bind les paramètres, puis `step`. Le SQL multi-statement est refusé dans cette forme (utilisez la version sans params pour ça). `SELECT` fonctionne mais les résultats sont jetés : utilisez `query` pour lire des lignes.

query : itérateur lazy `db:query` retourne un userdata callable. Utilisé dans un `for-in`, il rend une table par tour jusqu'à épuisement, puis termine naturellement :

```
for row in db:query("SELECT name FROM users") do
    print(row.name)
end
```

Une ligne est une table Lua indexée par nom de colonne. Un result set vide itère zéro fois, sans erreur.

On peut aussi appeler l'itérateur manuellement :

```
local iter = db:query("SELECT name FROM users")
local first = iter()          -- la première ligne, ou nil si vide
iter:close()                  -- libère tôt ; pas strictement nécessaire
```

Un `break` dans la boucle est OK : quand l'itérateur est ramassé par le GC, `__gc` finalise le statement sous-jacent.

Mapping des types Le même mapping est utilisé dans les deux sens (bind et lecture), avec deux asymétries (booléens et inférence BLOB) notées plus bas.

Type SQLite	Type Lua (lecture)	Valeur Lua bindable
NULL	absent de la table row	non bindable via params (voir plus bas)
INTEGER	integer	integer ; aussi <code>true</code> -> 1, <code>false</code> -> 0
REAL	number (float)	number non-integer
TEXT	string	string (défaut pour toutes les strings Lua)
BLOB	string (binary-safe)	non produit par bind en V1 (voir plus bas)

Booléens : `true/false` Lua sont bindés en INTEGER 1/0, puisque SQLite n’a pas de type BOOLEAN. À la lecture, la colonne revient en integer (pas en boolean) : le binding ne connaît pas votre intention.

Strings toujours bindées en TEXT en V1 : Lua n’a pas de type binaire séparé, et le TEXT SQLite préserve les octets NUL, donc pas de perte en pratique. Un futur helper `luapilot.sqlite.blob(data)` peut être ajouté si le bind BLOB strict devient utile.

NULL a deux limitations, toutes deux conséquences de la sémantique nil de Lua :

- *NULL n’est pas bindable via la table params* (Lua réduit `{ x = nil }` à `{}`, donc le bind ne voit jamais nil). Workaround : NULL directement dans le SQL, ou `COALESCE(?, NULL)` avec une valeur sentinelle.
- *Les colonnes NULL sont absentes de la table row*. `row.col == nil` fonctionne, mais `pairs(row)` saute les colonnes NULL (une table Lua ne peut pas stocker nil comme valeur). À documenter et tester en conséquence.

Noms de colonnes dupliqués (SELECT a, a FROM t ou conflits d’alias) fusionnent dans la table row ; la dernière valeur gagne. SQLite ne détecte pas ça au prepare. Utilisez des alias AS explicites en cas de doute.

Paramètres : positionnel et nommé `?` est positionnel ; `:name`, `@name`, `$name` sont tous des placeholders nommés valides (SQLite accepte les trois). La table Lua utilise le nom *sans préfixe* :

```
-- Positionnel
db:exec("INSERT INTO t VALUES (?, ?)", { 1, "hello" })

-- Nommé
db:exec("INSERT INTO t VALUES (:id, :name)",
      { id = 1, name = "hello" })

-- Mélange (rare mais autorisé)
db:exec("INSERT INTO t VALUES (?, :name, ?)",
      { 1, 3, name = "middle" })
```

Les mismatches font toujours raise (erreur de programmation, pas d’erreur SQL runtime) :

- *Param manquant* — INSERT (?, ?) VALUES avec `{ 1 }` : raise “missing positional param at index 2”.
- *Param en trop* — INSERT (?) VALUES avec `{ 1, 2 }` : raise “too many positional params”.
- *Nommé en trop* — :a avec `{ a = 1, zzz = "x" }` : raise “extra param ‘zzz’ (not used by this SQL)”.
- *Mauvais type* — `{ function() end }` : raise “cannot bind value of type ‘function’ at slot 1”.

Les erreurs SQL (colonne inconnue, contrainte violée, etc.) retournent `(nil, err)` à la place.

Transactions Pas de helper en V1. Utilisez BEGIN/COMMIT/ROLLBACK manuels :

```
db:exec("BEGIN")
for _, item in ipairs(items) do
  local ok, err = db:exec("INSERT INTO t VALUES (?, ?)",
    { item.id, item.name })

  if not ok then
    db:exec("ROLLBACK")
    return nil, err
  end
end
db:exec("COMMIT")
```

Un helper `transaction(fn)` arrivera peut-être en V2 si une API Lua propre se dégage (la sémantique `pcall` autour des erreurs est subtile).

Workers et concurrence LuaPilot n'ajoute *aucun lock global* autour de SQLite. La concurrence est gérée par les verrous fichier propres de SQLite :

- *Plusieurs readers* en parallèle : toujours OK.
- *Un writer à la fois* : SQLite sérialise les écritures ; les autres writers voient `SQLITE_BUSY` (refusé) sauf si `busy_timeout` leur permet un retry, ou si WAL est activé.
- *Mode WAL* permet readers + un writer en parallèle sur le même fichier, avec un meilleur throughput global. Recommandé pour toute app qui mélange lectures et écritures depuis plusieurs connexions.

Dans `luapilot.workers` : ne partagez pas un userdata `db` entre threads. La recommandation officielle SQLite est “*do not pass a database connection from one thread to another*” — même avec `SQLITE_THREADSafe=1`. Le pattern propre : soit chaque worker a son `open()`, soit un worker est désigné DB-owner et les autres lui parlent via la queue de messages des workers.

Cas limite : close pendant l'itération Si vous détenez un itérateur actif depuis `db:query` quand `db:close()` est appelé, l'itérateur *continue à marcher* jusqu'à épuisement. Le `sqlite3_close_v2` de SQLite marque le handle zombie et attend que le dernier statement soit finalisé avant de vraiment fermer. C'est la conception de SQLite, pas une curiosité LuaPilot :

```
local iter = db:query("SELECT id FROM t ORDER BY id")
db:close()
local row = iter()    -- rend toujours la première ligne
iter:close()          -- maintenant le handle DB est vraiment libéré
```

Pour une sémantique plus stricte, finalisez l'itérateur vous-même avant de fermer la base.

TOML

`luapilot.toml.decode` parse une chaîne TOML en table Lua. Le binding wrappe `toml++` (v1.0.0 de la spec TOML, header-only) et reflète le contrat d'erreur de `luapilot.json`.

```
local toml = luapilot.toml

local cfg, err = toml.decode([[
title = "My App"

[server]
host = "127.0.0.1"
port = 8080

[[users]]
name = "alice"
age = 30

[[users]]
name = "bob"
age = 25
]])

if not cfg then
    io.stderr:write("toml: ", err, "\n")
    return
end

print(cfg.title)           -- "My App"
print(cfg.server.host)     -- "127.0.0.1"
print(cfg.users[1].name)   -- "alice"
```

Mapping de types (TOML → Lua) :

Type TOML	Type Lua	Notes
string	string	UTF-8 préservé
integer	number (integer)	64 bits, sous-type integer Lua 5.5
float	number (float)	double IEEE 754
boolean	boolean	
array	table (séquence 1-indexée)	mimétisme du comportement <code>json.decode</code>
table	table (clés string)	incluant les formes dotted et <code>[section]/[[section]]</code>
local-date	string	ISO 8601 : "1979-05-27"
local-time	string	ISO 8601 : "07:32:00" (secondes fractionnaires optionnelles)
local-date-time	string	ISO 8601 : "1979-05-27T07:32:00"
offset-date-time	string	ISO 8601 : "1979-05-27T07:32:00Z" ou "...+02:00"

Sur les types temporels : TOML a quatre types date/time distincts ; Lua n'a pas de type date natif. On les rend comme strings ISO 8601 — avec perte (le tag de type original disparaît) mais prévisible. Re-parsez avec l'outil de votre choix si vous avez besoin d'un accès structuré.

Contrat d'erreur (strict miroir de `json.decode`) :

- **Succès** — (`table`, `nil`). Un document TOML a toujours une table à la racine ; la valeur renvoyée est donc toujours une table (possiblement vide pour un document vide ou un fichier contenant seulement des commentaires).
- **TOML invalide** — (`nil`, `"toml: <description> (line L, col C)"`). Inclut les clés redéfinies, les littéraux inconnus (`TRUE` n'est pas du TOML valide — seuls `true/false` en minuscules le sont), les chaînes non terminées, etc.
- **Mauvais usage** (argument manquant, type non-string) — lève via `luaL_error`. Contrairement à la plupart des fonctions LuaPilot acceptant des strings, `toml.decode` utilise `luaL_checktype(LUA_TSTRING)` strictement : passer un nombre lève au lieu de coercer.

```
-- Mauvais : tente de passer un nombre au parser TOML
local _, err = toml.decode(42)    -- error: bad argument
```

Hors v1 (tous additifs plus tard sous SemVer) :

- Pas d'`encode` — le mapping table-structurée-vers-sections de TOML a beaucoup de réponses défendables ; on préfère ne pas figer un défaut hasardeux. Utilisez `json.encode` pour les aller-retour machine-à-machine.
- Pas de `decode_file(path)` — utilisez `local s = assert(io.open(path)):read("a"); toml.decode(s)`.
- Pas de sentinelles (TOML n'a pas de `null`).
- Pas d'options de parsing — TOML est un format strict par design.

Voir les exemples pour aller plus loin...

sockets — client et serveur TCP

`luapilot.socket` fournit des sockets TCP bloquants avec timeouts — côté client (`connect`) comme côté serveur (`listen/accept`). POSIX pur sous le capot (zéro dépendance), des userdata avec `__gc` pour qu'un `close()` oublié ne fuie jamais un file descriptor.

```
local socket = luapilot.socket

-- Côté client
local cli, err = socket.connect("example.com", 80, 5)    -- timeout connect 5s
if not cli then
    io.stderr:write(err, "\n"); return
end
cli:set_timeout(2)                                       -- 2s par opération ensuite
cli:send("GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
local response = cli:recv_all()
cli:close()

-- Côté serveur (echo une ligne et ferme)
local srv = assert(socket.listen("127.0.0.1", 8080))
local peer = assert(srv:accept())
local line = peer:recv_line()
```

```
peer:send("you said: " .. line .. "\n")
peer:close()
srv:close()
```

Fonctions

Fonction	Renvoie	Notes
<code>socket.connect(host, port [, timeout])</code>	socket (nil, err)	Client TCP. <code>timeout</code> en secondes (float), s'applique uniquement à la phase de connexion.
<code>socket.listen(host, port [, backlog])</code>	socket (nil, err)	Fait <code>socket + setsockopt(SO_REUSEADDR) + bind + listen</code> en une fois. <code>host = ""</code> bind toutes les interfaces (AI_PASSIVE). Backlog par défaut : 16.

Méthodes (sur un socket connecté venant de `connect()` ou `accept()`) :

Méthode	Renvoie	Notes
<code>s:send(data, bytes, nil)</code>	(nil, "closed") (nil, err)	Envoie l'intégralité de <code>data</code> (boucle sur les writes partiels). Peer fermé → (nil, "closed"). Utilise MSG_NOSIGNAL pour qu'un peer fermé ne lève jamais SIGPIPE.
<code>s:recv(n, data, nil)</code>	(nil, "closed") (nil, "timeout") (nil, err)	Lit jusqu'à n octets (sémantique POSIX <code>read()</code> — peut renvoyer moins). EOF est la chaîne typée "closed".
<code>s:recv_line(data, nil)</code>	(nil, "closed", partial) (nil, "timeout")	Lit jusqu'à \n exclus. CRLF transparent (\r avant \n retiré). Si EOF en plein milieu, renvoie trois valeurs — ne pas perdre les octets déjà lus.
<code>s:recv_all(data, nil)</code>	(nil, "timeout") (nil, err)	Lit jusqu'à EOF du peer (la condition de succès normale pour celui-ci).
<code>s:set_timeout(timeout)</code>	(nil, err)	Secondes (float). 0 = pas de timeout (bloquer indéfiniment). S'applique à toutes les opérations bloquantes suivantes sur ce socket.
<code>s:close(true, nil)</code>		Idempotent (appeler sur un socket déjà fermé est OK).
<code>s:peer() {host, port}</code>	(nil, err)	Adresse distante de la connexion.
<code>s:sockname() {host, port}</code>	(nil, err)	Adresse locale du socket.

Méthodes sur un socket d'écoute (venant de `listen()`) :

Méthode	Renvoie	Notes
<code>s:accept()</code>	socket (nil, "timeout") (nil, err)	Accepte une connexion entrante. Renvoie un nouveau socket connecté. Honore <code>set_timeout</code> sur le socket d'écoute.
<code>s:close()</code> / <code>s:set_timeout(s)</code> / <code>s:sockname()</code>	—	Pareil que sur les sockets connectés.
<code>s:send</code> / <code>s:recv*</code>	(nil, err)	Échoue toujours : mauvaise direction.

Contrat d'erreur

- **Succès** — valeur (socket, byte count, data, ...) directement.
- **Échecs runtime attendus** sous forme de chaînes typées, renvoyés comme (nil, str) :
 - "closed" — le peer a fermé la connexion (EOF propre). Pour `recv_line` en milieu de ligne, les octets partiels arrivent comme 3e valeur de retour.
 - "timeout" — `set_timeout` a expiré.
 - "line too long" — `recv_line` refuse les lignes plus longues que 8 MiB. Un peer qui floode des octets sans newline ne peut plus faire grossir le buffer interne jusqu'à OOM. Les octets accumulés sont jetés avec cette erreur.
 - Les autres échecs de transport ont un préfixe "socket: <description>".

- **Mauvais usage** (argument manquant, mauvais type) — lève via `luaL_error`. Comme le reste de LuaPilot, `host` et `port` passent par `luaL_checkstring/luaL_checkinteger`, qui coercent silencieusement les nombres et les chaînes numériques. Passez une table pour forcer une erreur.

SO_REUSEADDR interne

`socket.listen` pose `SO_REUSEADDR` sur le socket avant `bind()`, inconditionnellement. C'est ce qui permet à un serveur qu'on vient de tuer de redémarrer sur le même port sans attendre ~60 secondes le `TIME_WAIT` — cas par défaut pour les tests, les serveurs de dev, et les petits daemons. Pas exposé dans l'API v1 ; on préfère avoir le comportement correct par défaut plutôt que de demander un flag à chaque fois.

Hors v1 (additif plus tard sous SemVer) :

- **UDP** — `socket.udp(...)` avec la même forme, quand le besoin émergera.
- **Mode non-bloquant** — `s:set_blocking(false)` plus une primitive `select/poll`. Très chevauchant avec le chantier `workers` ; le faire en standalone reviendrait à livrer 30% de `workers` sous un autre nom.
- **Options avancées** — `keepalive`, `nodelay`, `SO_REUSEPORT`, `SO_LINGER`, tailles de buffer custom, etc. Ajoutées à la demande.
- **bind() exposé séparément** — `listen` fait tout-en-un par design (décision SOCK-2). Exposer `bind` brut est possible mais aucun cas d'usage ne le requiert encore.

Pour les sockets clients TLS (`connect_tls` et `starttls`), voir la section suivante.

Voir les exemples pour aller plus loin...

sockets TLS — connect_tls et starttls

`luapilot.socket` fournit aussi des sockets clients TLS, partageant le même userdata et les mêmes méthodes (`send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `close`, `peer`, `sockname`) que le TCP brut. TLS est une variante de TCP, pas un module séparé — une fois connecté, le socket se comporte à l'identique.

```
local socket = luapilot.socket

-- Connexion TLS directe (IRC sur TLS sur Libera.Chat, etc.)
local s, err = socket.connect_tls("irc.libera.chat", 6697,
    { timeout = 5 })
if not s then io.stderr:write(err, "\n"); return end
s:set_timeout(30)
s:send("NICK my-bot\r\nUSER my-bot 0 * :LuaPilot bot\r\n")
while true do
    local line, e = s:recv_line()
    if not line then break end
    print(line)
end
s:close()

-- Pattern STARTTLS (SMTP, IMAP, XMPP, IRC avec CAP STARTTLS, etc.)
local plain, err = socket.connect("smtp.example.com", 587, 5)
plain:set_timeout(10)
plain:recv_line() -- lire le greeting serveur
plain:send("EHLO me.example.com\r\n")
-- (lire la réponse EHLO, envoyer STARTTLS, lire la réponse OK...)
local ok, terr = plain:starttls({ hostname = "smtp.example.com" })
if not ok then io.stderr:write(terr, "\n"); return end
-- À partir d'ici, plain est chiffré. Renvoyer le prochain EHLO via TLS.
plain:send("EHLO me.example.com\r\n")
```

Fonctions et méthodes

Appel	Renvoie	Notes
<code>socket.connect_tls(host, port [, opts])</code>	<code>(socket, err)</code>	Connect TCP + handshake TLS en un appel. Miroir de <code>connect</code> .

<code>s:starttls([opts])</code>	<code>(true, nil)</code> <code>(nil, err)</code>	Promeut un socket TCP connecté existant en TLS sur place. Utilisé pour les protocoles qui négocient TLS sur du plaintext (SMTP/IMAP STARTTLS, etc.).
---------------------------------	---	--

Table d'options opts (tous les champs optionnels) :

Champ	Type	Défaut	Signification
<code>timeout</code>	number	aucun (secondes)	Deadline par appel. S'applique à la phase connect et au handshake TLS. Après l'établissement du socket, utiliser <code>s:set_timeout()</code> pour les reads/writes suivants.
<code>verify</code>	boolean	<code>true</code>	Si <code>true</code> , le certificat peer est validé contre le trust store et contre le hostname attendu. Si <code>false</code> , le handshake TLS se termine quelle que soit la validité du cert — à n'utiliser que pour le debug, les serveurs de test auto-signés, ou un opt-out explicite.
<code>ca_cert</code>	string	aucun (chemin)	Chemin vers un fichier de certificat CA encodé PEM. Utilisé quand le peer est signé par une CA custom non présente dans le trust store système.
<code>ca_path</code>	string	aucun (chemin)	Chemin vers un répertoire de certificats CA PEM (format OpenSSL <code>c_rehash</code>). Moins courant que <code>ca_cert</code> .
<code>hostname</code>	string	<code>host</code> (pour <code>connect_tls(hostname="api.example.com")</code>)	Hostname utilisé pour SNI et pour le check CN/SAN du certificat. À override pour se connecter par IP en vérifiant un vhost (ex. <code>connect_tls("1.2.3.4", 443, hostname="api.example.com")</code>).
<code>min_version</code>	string	"1.2"	Version TLS minimum. "1.2" ou "1.3". La négociation prend toujours la plus haute mutuellement supportée (TLS 1.3 si les deux côtés sont d'accord).

Important : starttls et verify. Contrairement à `connect_tls`, `starttls` n'a pas de hostname implicite (le socket vient d'un `connect` précédent qui a pu utiliser une IP). Quand `verify=true` (défaut), il faut **obligatoirement** passer `opts.hostname` explicitement. L'appel refuse sinon avec `tls: starttls with verify=true requires opts.hostname`. Cela évite la vérification silencieuse "chain only" que certaines libs adoptent par défaut, et qui est plus faible qu'elle n'en a l'air.

Contrat d'erreur (cohérent avec les sockets bruts) :

- **Succès** — valeur (socket, byte count, etc.) directement.
- **Erreurs typées en chaîne** comme `(nil, str)` :
 - "closed" — le peer a fermé la session TLS (close_notify reçu).
 - "timeout" — deadline par appel expirée.
- Les **autres erreurs** sont préfixées "tls: ". Les échecs de validation du certificat sont explicites : par exemple, `tls: certificate verify failed: certificate has expired`, `tls: certificate verify failed: Hostname mismatch`, `tls: certificate verify failed: self-signed certificate`.
- **Mauvais usage** (mauvais types d'arg, arguments manquants) lève via `luaL_error` comme partout ailleurs dans LuaPilot. Passez une table pour forcer une erreur sur `host/port` (les chaînes numériques sont coercées silencieusement — même convention que le reste de LuaPilot).

Politique de vérification de certificat. Strict par défaut (`verify=true`) : la chaîne est validée contre le trust store système, et le CN/SAN du certificat doit correspondre à `host` (ou `opts.hostname` si fourni). Pour désactiver entièrement la vérification, passer explicitement `verify=false` — il n'y a pas d'entre-deux (pas de mode "chain only, no hostname").

Trust store système. LuaPilot embarque sa propre OpenSSL, donc il n'hérite pas automatiquement de la config `ca-certificates` de la distro. Pour que `verify=true` fonctionne en l'état, deux mécanismes complémentaires sont en place :

1. L'OpenSSL embarquée est compilée avec `--openssldir=/etc/ssl`, ce qui couvre Arch, Debian, Ubuntu, Alpine, Gentoo, et la plupart des distros qui suivent le chemin `/etc/ssl/certs/`.
2. À l'init TLS, LuaPilot sonde une liste d'emplacements de CA bundles connus et charge le premier qui existe. Couvre Fedora, RHEL, CentOS, Rocky, Alma, OpenSUSE, FreeBSD, NetBSD.

Si aucun des deux ne trouve un CA store, l'init TLS réussit quand même mais `verify=true` échouera au handshake (le message d'erreur nommera le peer non signé). Override le trust store avec `ca_cert` ou `ca_path`, ou positionne les variables d'environnement `SSL_CERT_FILE` / `SSL_CERT_DIR` (lues par le mécanisme de `verify` paths par défaut d'OpenSSL).

Versions TLS. TLS 1.2 minimum par défaut ; TLS 1.3 négocié automatiquement si les deux côtés le supportent. Les protocoles anciens (SSL 3, TLS 1.0, TLS 1.1) sont inconditionnellement rejetés — ils sont cassés, et aucun serveur moderne ne devrait s'y reposer.

Hors v1 (additif plus tard sous SemVer) :

- **TLS côté serveur** — `socket.listen_tls()` et le chargement de certificat/clé correspondant. Reporté parce qu'ajouter du TLS serveur ouvre six ou sept décisions de design (PEM/DER, chemin de fichier vs in-memory, gestion de passphrase, hot-reload de certificat...) qui méritent leur propre passe de design.
- **Certificats clients (mutual TLS)** — seule la vérification côté serveur est supportée aujourd'hui ; les clients ne peuvent pas présenter leur propre certificat.
- **ALPN** — nécessaire pour la négociation HTTP/2 ; pas pour IRC, SMTP, IMAP, XMPP, ou d'autres protocoles texte.
- **Session resumption** — une optimisation de performance, pas un manque fonctionnel.
- **DTLS** — TLS sur UDP.

Voir les exemples pour aller plus loin...

inotify — surveillance de système de fichiers

`luapilot.inotify` enveloppe `inotify(7)`, le mécanisme du noyau Linux qui notifie un programme dès qu'un fichier ou un répertoire change — création, écriture, suppression, déplacement. C'est l'outil propre pour réagir à un événement *au lieu* de scruter un dossier en boucle : réveil quasi-instantané, zéro CPU au repos. POSIX/Linux direct, zéro dépendance externe (comme `socket` et `signal`), et un userdata avec `__gc` pour qu'un `close()` oublié ne fuie jamais de file descriptor.

```
local inotify = luapilot.inotify

local w = assert(inotify.new())

-- Surveiller un dossier de dépôt. La liste d'événements est
-- OBLIGATOIRE et explicite - pas d'implicite « tout ».
local wd, err = w:add("/srv/incoming", { "close_write", "moved_to" })
if not wd then
    io.stderr:write(err, "\n"); return
end

-- Boucle de surveillance. read() bloque jusqu'à un événement (ou
-- le timeout), et rend un ARRAY d'événements (le noyau en batche
-- plusieurs).
while true do
    local events, rerr = w:read(5)          -- timeout 5s
    if not events then
        if rerr == "timeout" then
            -- rien en 5s : occasion de faire autre chose, puis reboucler
        elseif rerr == "interrupted" then
            break                                -- un signal géré est arrivé (SIGTERM...)
        else
            io.stderr:write("inotify: ", rerr, "\n"); break
        end
    else
        for _, ev in ipairs(events) do
            if ev.events.overflow then
                -- la queue noyau a débordé : des événements ont été
                -- perdus, il faut rescanner le dossier soi-même.
                rescan("/srv/incoming")
            elseif ev.events.close_write or ev.events.moved_to then
                process_new_file("/srv/incoming/" .. ev.name)
            end
        end
    end
end
end
```

`w.close()`

Pourquoi `close_write` / `moved_to` plutôt que `create` : surveiller la *fin* d'écriture (`close_write`) ou l'*arrivée* par renommage (`moved_to`) garantit un fichier complet. Un `create` se déclenche dès l'ouverture, avant que l'écrivain ait fini — on lirait un fichier tronqué. Si le producteur écrit dans un `.tmp` puis fait `rename()` vers le nom final (pattern recommandé), `moved_to` est le bon événement à écouter.

Fonctions

Fonction	Renvoie	Notes
<code>inotify.new()</code>	<code>(watcher (nil, err))</code>	Crée une instance inotify (un FD noyau). Échec runtime (trop de FD/instances) → <code>(nil, err)</code> .

Méthodes (sur un watcher venant de `new()`) :

Méthode	Renvoie	Notes
<code>w.add(path, (wd, nil) (nil, err) events [, opts])</code>		Pose une surveillance. events = liste obligatoire et non vide de noms (voir table ci-dessous). Renvoie le watch descriptor (integer). opts.onlydir = true → échoue si <code>path</code> n'est pas un répertoire.
<code>w.read([timeout], (events, nil) (nil, "timeout") (nil, "interrupted") (nil, err))</code>		Lit les événements en attente. timeout en secondes (float) ; absent = bloquant infini , 0 = non bloquant (rend ce qui est dispo tout de suite). Renvoie un array d'événements batchés.
<code>w.remove(wd, (true, nil) (nil, err))</code>		Retire une surveillance par son watch descriptor. Le noyau émet un événement ignored pour ce <code>wd</code> juste après — visible au prochain <code>read()</code> .
<code>w.close()</code>	<code>(true, nil)</code>	Ferme le FD inotify. Idempotent. Filet <code>__gc</code> si oublié.

Forme d'un événement (une entrée de l'array renvoyé par `read()`) :

Champ	Type	Notes
<code>wd</code>	integer	Le watch descriptor d'origine (-1 si c'est un événement d'overflow).
<code>name</code>	string	L'entrée concernée dans le répertoire surveillé. "" si l'événement vise la cible elle-même (ex. <code>delete_self</code>).
<code>events</code>	table	Le masque décodé en flags lisibles, ex. { <code>close_write = true</code> }. Peut en cumuler plusieurs.
<code>is_dir</code>	boolean	true si l'entrée concernée est un répertoire (IN_ISDIR).
<code>cookie</code>	integer	0 la plupart du temps ; non nul pour apparier un <code>moved_from</code> et un <code>moved_to</code> d'un même renommage.

Événements disponibles

Noms passables à `add()` (et présents dans `ev.events` à la lecture) :

Nom	Déclenché par
<code>access</code>	Lecture du fichier.
<code>modify</code>	Écriture dans le fichier.
<code>attrib</code>	Changement de métadonnées (permissions, mtime, owner...).
<code>close_write</code>	Fermeture d'un fichier qui était ouvert en écriture.
<code>close_nowrite</code>	Fermeture d'un fichier ouvert en lecture seule.
<code>close</code>	Raccourci : <code>close_write</code> ou <code>close_nowrite</code> .
<code>open</code>	Ouverture du fichier.
<code>moved_from</code>	Une entrée a été déplacée <i>hors</i> du répertoire surveillé.
<code>moved_to</code>	Une entrée a été déplacée <i>vers</i> le répertoire surveillé.
<code>move</code>	Raccourci : <code>moved_from</code> ou <code>moved_to</code> .
<code>create</code>	Création d'une entrée dans le répertoire surveillé.
<code>delete</code>	Suppression d'une entrée du répertoire surveillé.

Nom	Déclenché par
<code>delete_self</code>	La cible surveillée elle-même a été supprimée.
<code>move_self</code>	La cible surveillée elle-même a été déplacée.

Deux flags supplémentaires apparaissent dans `ev.events` sans être demandables dans `add()` (le noyau les pose tout seul) :

Nom	Signification
<code>ignored</code>	La surveillance a été retirée — par <code>remove()</code> , ou parce que la cible a été supprimée / son FS démonté.
<code>unmount</code>	Le système de fichiers portant la cible a été démonté.
<code>overflow</code>	La queue d'événements du noyau a débordé : des événements ont été perdus. Voir ci-dessous.

Overflow : à ne jamais ignorer. Si les événements arrivent plus vite que vous ne lisez (`read()`), le noyau finit par déborder sa file interne et émet un événement spécial `{ wd = -1, events = { overflow = true } }`. C'est le seul cas où `inotify` perd silencieusement des notifications. Quand vous le voyez, la seule réponse correcte est de **rescanner vous-même** le répertoire surveillé pour rattraper l'état réel — les événements perdus ne reviendront pas. C'est pourquoi LuaPilot le remonte explicitement plutôt que de l'avalier.

Contrat d'erreur (cohérent avec `socket` / `http` / `toml`) :

- **Mauvais usage** (mauvais *type* d'argument, `events` absent ou non-table) → lève via `luaL_error`, comme partout dans LuaPilot.
- **Mauvaise valeur / erreur runtime** → `(nil, err)` : liste d'événements vide, nom d'événement inconnu, chemin inexistant, watch descriptor invalide, watcher déjà fermé, etc.
- **Action réussie** (`remove`, `close`) → `(true, nil)`.

Interruption par signal. Comme `socket:recv` et `luapilot.sleep` : si un signal géré par `luapilot.signal` (ex. `SIGTERM` envoyé par `systemctl stop`) arrive pendant un `w:read()` bloquant, l'appel renvoie immédiatement `(nil, "interrupted")` et votre callback de signal s'exécute avant le retour. C'est ce qui permet d'arrêter proprement un watcher de longue durée sans attendre l'expiration du timeout.

Hors v1 (additif plus tard sous SemVer) :

- **Surveillance récursive** — `inotify(7)` lui-même n'est pas récursif : un watch couvre un seul répertoire, pas son sous-arbre. La récursion propre (*tree-walk* + ajout dynamique de watches à la création d'un sous-dossier + gestion fine de l'overflow) est un vrai chantier de design ; elle est constructible en Lua pur par-dessus l'API actuelle.
- **Modifieurs `add()` supplémentaires** : `dont_follow` (ne pas déréférencer un symlink), `oneshot` (un seul événement puis retrait auto), `mask_add` (cumuler avec un masque existant). v1 expose le minimum utile (`onlydir`).
- **fanotify** — surveillance à l'échelle d'un point de montage, nécessite `CAP_SYS_ADMIN` : autre API, autre cas d'usage.

Voir les exemples pour aller plus loin...

workers — jobs parallèles

`luapilot.workers` exécute du code Lua en parallèle via des threads OS. Chaque worker a son **propre lua_State** et sa **propre mémoire** — pas d'état partagé, pas de race conditions côté Lua. La communication passe par des **messages sérialisés** (JSON en interne). C'est le modèle Erlang / Web Workers : isoler, envoyer un message, joindre.

Contrairement aux langages avec un verrou global d'interpréteur (le GIL Python), les workers LuaPilot tournent vraiment sur plusieurs cœurs CPU. Spawnez N workers et vous pouvez utiliser N cœurs.

L'API supporte **deux patterns d'usage** :

- **Fork-join** — spawn d'un worker qui calcule un résultat et meurt. On utilise `:join()` ou `:poll()` pour récupérer la valeur de retour.
- **Persistant** — spawn d'un worker qui boucle sur `worker.recv()`, traite les messages et répond via `worker.send()`. On utilise les méthodes côté parent `w:send()` / `w:recv()` / `w:close()` pour communiquer.

Les deux patterns partagent le même `spawn()` et le même modèle d'isolation. Le choix est dans le code du worker : un `return` au top-level c'est fork-join ; une boucle `while ... worker.recv()` c'est persistant.

Fork-join : calculer un résultat

```
local workers = luapilot.workers

-- Trivial : spawn d'un worker, on attend le résultat
local w = workers.spawn("return 1 + 2")
local ok, value = w:join()    -- ok=true, value=3

-- Passer des arguments au worker
local w2 = workers.spawn(
    "return worker.args.x * worker.args.y",
    { x = 6, y = 7 })
local ok, value = w2:join()    -- ok=true, value=42

-- Pattern canonique : fan-out de N jobs CPU-bound
local urls = { "http://a/", "http://b/", "http://c/", "http://d/" }
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = workers.spawn([
        local url = worker.args.url
        local r, err = luapilot.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end
-- Join tout (bloque jusqu'à ce que chacun finisse). Wall-clock total
-- ~ le plus lent des fetches, pas leur somme.
for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end
```

Persistant : messagerie bidirectionnelle

```
-- Worker qui fait écho à chaque message reçu, avec un marqueur.
local w = workers.spawn([
    while true do
        local ok, msg = worker.recv() -- bloque sur l'entrée parent
        if not ok then break end      -- "closed" : le parent ferme
        worker.send({ echo = msg })
    end
    return "exited cleanly"
])

w:send("hello")
w:send("world")
w:send(42)

local _, r1 = w:recv()    -- (true, { echo = "hello" })
local _, r2 = w:recv()    -- (true, { echo = "world" })
local _, r3 = w:recv()    -- (true, { echo = 42 })

-- Signaler la fin de l'entrée. Le worker.recv() côté worker rend
-- (false, "closed"), il sort de sa boucle, la valeur de retour
-- traverse via join().
w:close()
local ok, value = w:join()    -- (true, "exited cleanly")
```

Cas d'usage typiques d'un worker persistant : un worker logger, un worker connexion DB, un handler de protocole

long-running (IRC, WebSocket), ou tout ce qui doit survivre à une requête unique.

Référence API

Appel	Renvoie	Notes
<code>workers.spawn(<i>code</i> [, <i>args</i>] [, <i>opts</i>])</code>	(<i>nil</i> , <i>err</i>)	Démarre un worker. <code>opts.inbox_capacity</code> et <code>opts.outbox_capacity</code> fixent les capacités des queues bornées (défaut 64 chacune). Renvoie (<i>nil</i> , <i>err</i>) si la sérialisation échoue (<i>function</i> , <i>userdata</i> , <i>coroutine</i> , <i>cycle</i> , NaN/Inf, non-UTF-8) ou si <code>pthread_create</code> échoue.
<code>w:join()</code>	(<i>ok</i> , <i>value</i>)	Bloque jusqu'à la fin du worker. <i>ok=true</i> , <i>value</i> =<valeur de retour> en succès ; <i>ok=false</i> , <i>value</i> =<message d'erreur> si le worker a levé.
<code>w:poll()</code>	(<i>state</i> , <i>value</i>)	Non bloquant , état du worker. <i>state</i> vaut "running", "done", ou "error". <i>value</i> est nil tant que running, la valeur de retour en done, le message d'erreur en error.
<code>w:send(<i>value</i> (<i>true</i>, [, <i>nil</i>] [<i>false</i>, <i>reason</i>]) [, <i>timeout</i>])</code>	(<i>true</i> , <i>nil</i>) (<i>false</i> , <i>reason</i>)	Push un message dans l'inbox du worker. Bloque si pleine ; <i>timeout</i> en secondes. <i>reason</i> ∈ "full" (timeout=0 et queue pleine), "timeout" (timeout>0 expiré), "closed".
<code>w:recv(<i>[timeout]</i> [, <i>true</i>, <i>value</i>] [<i>false</i>, <i>reason</i>])</code>	(<i>true</i> , <i>value</i>) (<i>false</i> , <i>reason</i>)	Pop un message depuis l'outbox du worker. Bloque si vide ; <i>timeout</i> en secondes. <i>reason</i> ∈ "empty" (timeout=0 et queue vide), "timeout", "closed".
<code>w:close()</code>	(<i>true</i> , <i>nil</i>)	Ferme l'inbox du worker. Les <code>w:send</code> ultérieurs rendent (<i>false</i> , "closed") ; le <code>worker.recv()</code> côté worker rend (<i>false</i> , "closed") pour qu'il sorte proprement de sa boucle. Idempotent.

Côté worker :

Appel	Renvoie	Notes
<code>worker.send(<i>value</i> [, <i>timeout</i>])</code>	(<i>true</i> , <i>nil</i>) (<i>false</i> , <i>reason</i>)	Push un message vers le parent (direction outbox). Bloque si l'outbox est pleine (backpressure).
<code>worker.recv(<i>[timeout]</i> [, <i>true</i>, <i>value</i>] [<i>false</i>, <i>reason</i>])</code>	(<i>true</i> , <i>value</i>) (<i>false</i> , <i>reason</i>)	Pop un message depuis le parent (direction inbox). Bloque si vide.
<code>worker.args</code>	table nil	La table d'args passée au <code>spawn()</code> , ou nil.

Conventions de retour La convention varie entre **côté parent** et **côté worker** :

- **Côté parent** `w:send` / `w:recv` / `w:join` / `w:close` suivent la convention pcall-style (*ok*, *value_or_reason*) — *value* peut légitimement valoir *nil* après un échange réussi, donc (*false*, "...") est la seule façon de signaler un échec sans ambiguïté.
- **Côté parent** `w:send` avec une valeur non sérialisable rend (*nil*, *err*) au lieu de (*false*, *err*) — c'est la convention (*result*, *err*) utilisée partout ailleurs dans LuaPilot quand une *mauvaise valeur* est passée à la frontière de l'API (cf. `luapilot.json.encode`).
- **Côté worker** `worker.send` / `worker.recv` utilisent systématiquement (*ok*, *value_or_reason*) — sans exception. À l'intérieur d'un worker on écrit toujours `local ok, msg = worker.recv(); if not ok then ... end`.

Important : `w:close()` n'interrompt pas `w:close()` effectue une **fermeture logique** de l'inbox du worker — il **n'interrompt pas** le worker s'il est actuellement bloqué dans une opération longue (un `socket.recv()` lent, un long sleep, un `http.get()` long, etc.). Le worker ne voit la fermeture que la *fois suivante* où il appelle `worker.recv()`.

```
-- ANTI-PATTERN : ce worker n'est pas interruptible
local w = workers.spawn([
    local sock = luapilot.socket.connect("slow-server", 80, 30)
    sock:set_timeout(60)
    sock:recv(4096) -- bloque jusqu'à 60 secondes, ignore w:close()
    return "done"
])
```



```
w:close()  -- le worker continue à lire sur la socket ; ça n'affecte
           -- que l'inbox, que le worker ne lit pas
w:join()   -- attend jusqu'à 60 secondes
```

Pour qu'un worker soit responsif au shutdown, **écrivez-le coopérativement** : intercalez des `worker.recv(0)` entre les opérations longues et sortez de la boucle quand vous voyez (`false`, `"closed"`) :

```
-- PATTERN COOPÉRATIF
local w = workers.spawn([[
  while true do
    -- Check non bloquant rapide : le parent veut-il qu'on s'arrête ?
    local ok, msg = worker.recv(0)
    if not ok and msg == "closed" then break end
    if ok then
      -- traiter msg
    end

    -- Faire une unité de travail (courte et bornée). NE PAS faire
    -- une opération qui pourrait bloquer plus longtemps que la
    -- latence de shutdown acceptable.
    do_one_step()
  end
  return "exited cleanly"
]])
```

`:kill()` est intentionnellement **absent** de l'API v1 : terminer de force un worker en milieu de pthread laisse l'état partagé (file descriptors, mutexes, contextes OpenSSL) dans des états non récupérables. Le shutdown coopératif est le seul modèle sûr. Si un worker doit pouvoir interrompre une opération socket longue, posez `set_timeout()` à la latence d'arrêt acceptable.

Sérialisation des arguments et messages Les arguments passés au `spawn()` et les messages échangés via `send/recv` doivent être représentables en JSON :

Type Lua	Supporté ?	Notes
<code>nil</code> , <code>boolean</code> , <code>string</code> , <code>number</code>	oui	Chaînes UTF-8 uniquement. NaN et Infinity refusés.
<code>table</code> (séquence 1..n)	oui	Sérialisée comme array JSON.
<code>table</code> (clés string)	oui	Sérialisée comme object JSON.
<code>table</code> (mixte / trouée / clés non-string)	non	Refusée à la sérialisation.
<code>function</code> , <code>userdata</code> , <code>coroutine</code>	non	Refusés. Vous ne pouvez pas passer une socket ouverte, un file handle, etc. à ou depuis un worker.
Tables cycliques	non	Refusées dès la première détection de cycle.

Caveat integer / float. JSON n'a qu'un seul type numérique, donc la distinction integer/float est perdue au transfert. Un worker qui renvoie 42 (integer) peut arriver comme 42.0 (float) côté parent, ou vice-versa. Si votre code teste `math.type(v) == "integer"`, convertissez explicitement avec `math.tointeger(v)` après le join ou le recv.

Ce qu'il y a à l'intérieur d'un worker Chaque worker reçoit un `lua_State` neuf avec :

- La bibliothèque standard Lua (`io`, `os`, `string`, `table`, `math`, `coroutine`, `debug`, `package`).
- L'API complète `luapilot.*` (`http`, `socket`, `json`, `toml`, `sys`, `helpers` filesystem, hashes, etc.).
- Les modules bundlés via `require()` (`inspect`, `argparse`, `logging`).
- Les modules utilisateur via `require()` fonctionnent comme dans le parent (résolus depuis le répertoire du script, ou depuis le ZIP embarqué en mode packagé).

Deux globales sont posées différemment du parent :

- `worker` est une table contenant `args`, `send`, et `recv`. `worker.args` contient la table passée au `spawn` (ou `nil`).
- `arg` vaut `nil` (le worker n'hérite pas du vecteur `arg` du parent). Utilisez `worker.args` à la place.

Cycle de vie et queues bornées Les deux queues sont **bornées**. Les valeurs par défaut sont 64 entrées chacune (`inbox_capacity` et `outbox_capacity` au `spawn`). Quand une queue est pleine, le `send` correspondant bloque (ou rend `"full"` / `"timeout"` selon l'argument `timeout`). C'est de la **backpressure** intentionnelle — une queue non bornée grossirait silencieusement jusqu'à l'épuisement mémoire si le consommateur est plus lent que le producteur.

Quand le worker se termine (return ou erreur), son outbox `worker.send` est **drainée d'abord** : les messages en attente restent visibles via `w:recv()` jusqu'à ce que la queue soit vide, puis `w:recv()` rend (`false`, `"closed"`). Symétriquement, un `w:send` après mort du worker rend (`false`, `"closed"`) immédiatement.

Si le parent n'appelle jamais `w:join()` ni `w:poll()`, le `__gc` du userdata join la thread automatiquement (en fermant les deux queues pour débloquer toute opération pendante). C'est un filet de sécurité, pas un substitut à une gestion explicite du lifecycle.

Important : le `__gc` BLOQUE si le worker est occupé. Quand le garbage collector Lua finalise un handle de worker abandonné, il appelle `pthread_join()` sur la thread du worker. Si le worker est en train d'exécuter une opération longue qui ne vérifie pas régulièrement son inbox (un `socket:recv()` lent avec timeout long, un `http.get()` synchrone contre un serveur lent, un `luapilot.sleep` de 60 secondes, etc.), le GC — et donc **l'ensemble du processus LuaPilot** — se fige jusqu'au retour du worker. C'est cohérent avec la décision « pas de `:kill()` en v1 » : le shutdown coopératif est le seul modèle sûr.

En pratique, cela signifie :

- **Toujours garder une référence** aux workers spawnés et faire explicitement `w:close() + w:join()` à la sortie du programme. Ne pas compter sur le garbage collection pour le shutdown.
- **Écrire les workers de manière coopérative** (cf. la section « `w:close()` n'interrompt pas » plus haut) — des opérations courtes et bornées, entrelacées avec des vérifications `worker.recv(0)`.
- **Borner les appels bloquants dans les workers** : préférer `sock:set_timeout(5)` au timeout infini par défaut, pour qu'un `recv()` ne puisse pas bloquer plus longtemps qu'un maximum connu.

Hors v1 (additif plus tard sous SemVer)

- `spawn(function, args)` via `string.dump`. La forme actuelle basée sur string couvre déjà tous les cas d'usage ; la forme fonction serait plus ergonomique mais a des subtilités (les upvalues ne sont pas capturés, seulement les locals au top-level).
- `:kill()` — terminer de force un worker (cf. « n'interrompt pas » plus haut).
- **Pool de workers persistants** — implémentable en Lua pur au-dessus de `spawn`. Un `workers.pool(N)` bundlé pourra arriver plus tard.
- `workers.channel()` — channels indépendants pour communication worker-à-worker. Le modèle actuel est « routage par le parent uniquement ».
- **Sérialisation binaire** — JSON suffit pour les payloads typiques. Un format binaire plus rapide pourra être ajouté si un cas d'usage avec gros volumes émerge.

Contribuer

Les contributions sont bienvenues ! Ouvrez une issue ou une pull request pour toute suggestion ou amélioration.

Licence

Ce projet est sous licence MIT. Voir le fichier LICENSE pour les détails.

Note : Cette version française est maintenue en parallèle de `README.md` (référence). En cas de divergence ponctuelle, c'est la version anglaise qui fait foi.